

Autenticación y Seguridad

Protección de Contraseñas y Login

Python Full Stack - Clase 31

¿Qué aprenderemos hoy?

-  Entender qué es hashing y encriptación
-  Aprender técnicas para proteger contraseñas
-  Implementar Bcrypt en Flask
-  Crear sistema de registro con contraseñas seguras
-  Implementar inicio de sesión completo
-  Crear funcionalidad de cierre de sesión
-  Practicar con un proyecto completo de autenticación

Protección de Contraseñas

El Problema de Seguridad

¿Qué pasa si guardamos las contraseñas directamente en la base de datos?

Usuario: juan@email.com

Password: miPassword123

Se guarda en BD: miPassword123 ← ¡PELIGROSO!

Si un hacker accede a la BD:

- Ve todas las contraseñas
- Puede iniciar sesión como cualquier usuario
- Accede a información privada

¡NUNCA debemos guardar contraseñas en texto plano! 

¿Qué es Hashing?

Hashing es una técnica que convierte datos en una cadena de caracteres que **no se puede revertir**.

Características:

-  **Irreversible:** No puedes obtener la contraseña original del hash
-  **Determinístico:** La misma contraseña siempre produce el mismo hash
-  **Único:** Diferentes contraseñas producen diferentes hashes

Hashing vs Encriptación

Característica	Hashing	Encriptación
Reversible	✗ No	✓ Sí (con clave)
Uso	Contraseñas	Datos sensibles (tarjetas)
Clave necesaria	✗ No	✓ Sí
Ejemplo	Contraseñas de usuarios	Números de tarjeta de crédito

Regla de oro:

- Contraseñas → Hashing (nunca necesitamos verlas)
- Datos que necesitamos leer → Encriptación (tarjetas, mensajes)

El Problema con Hash Simple

Hash simple (MD5, SHA1):

Contraseña: "password123"

Hash MD5: 482c811da5d5b4bc6d497ffa98491e38

Problemas:

- ✗ Algoritmos antiguos ya fueron "resueltos"
- ✗ Tablas arcoíris (rainbow tables) con hashes comunes
- ✗ Ataques de fuerza bruta muy rápidos

La Solución: Salt

Salt es una cadena aleatoria que se agrega a la contraseña antes de hacer hash.

Proceso:

```
Contraseña: "password123"  
Salt: "a8f5f167f44f4964e6c998dee827110c"  
Concatenación: "password123a8f5f167f44f4964e6c998dee827110c"  
Hash: "9f86d081884c7d659a2feaa0c55ad015..."
```

Ventaja: Cada usuario tiene su propio salt único, haciendo más difícil los ataques.

Salt por Usuario

Mejor práctica: Cada usuario tiene su propio salt almacenado en la BD.

```
Usuario 1:  
  Salt: "abc123..."  
  Password: "password123"  
  Hash: "hash1..."  
  
Usuario 2:  
  Salt: "xyz789..."  
  Password: "password123" (misma contraseña)  
  Hash: "hash2..." (hash diferente!)
```

Ventaja: Aunque dos usuarios tengan la misma contraseña, los hashes son diferentes.

La Solución Final: Bcrypt

Bcrypt es un algoritmo diseñado específicamente para ser **lento**.

Características:

-  Genera salt automáticamente
-  Almacena salt en el hash
-  Es intencionalmente lento (ralentiza ataques)
-  Configurable (más rondas = más seguro pero más lento)

Ventaja: Hace que los ataques de fuerza bruta sean prácticamente imposibles.

¿Cómo Funciona Bcrypt?

Estructura del hash de Bcrypt:

`$2b$12$LNvGzmb593PlalHy.2vduixf2a90FdPAy5cy1KnCGaYKGESzsvWq`

The diagram illustrates the structure of the bcrypt hash `$2b$12$LNvGzmb593PlalHy.2vduixf2a90FdPAy5cy1KnCGaYKGESzsvWq`. It uses vertical and horizontal lines to segment the string into four parts, each with a label to its right:

- Hash (31 caracteres)**: Points to the final 31 characters of the hash, `1KnCGaYKGESzsvWq`.
- Salt (22 caracteres)**: Points to the 22-character salt, `LNvGzmb593PlalHy.`
- Rondas (12 = 2¹² iteraciones)**: Points to the number of rounds, `12`.
- Versión (2b)**: Points to the version identifier, `2b`.

Componentes:

- `2b`: Versión de Bcrypt
- `12`: Número de rondas ($2^{12} = 4096$ iteraciones)
- Salt: Generada automáticamente
- Hash: Resultado final

Flask - Bcrypt

Instalación de Flask-Bcrypt

Paso 1: Instalar el paquete

```
pipenv install Flask-Bcrypt
```

```
pip install Flask-Bcrypt
```

Paso 2: Importar y configurar en tu aplicación

```
from flask_bcrypt import Bcrypt
from flask_app import app

bcrypt = Bcrypt(app) # Generamos un objeto bcrypt
```

Nota: El objeto `bcrypt` se crea una vez y se usa en toda la aplicación.

Métodos de Bcrypt

Bcrypt tiene dos métodos principales:

1. `generate_password_hash(contraseña)` :

- Recibe: Contraseña en texto plano
- Retorna: Contraseña hasheada (lista para guardar en BD)

2. `check_password_hash(hash, contraseña)` :

- Recibe: Hash almacenado y contraseña a verificar
- Retorna: `True` si coinciden, `False` si no

Hashear Contraseña en el Registro

```
from flask_bcrypt import Bcrypt
from flask_app import app

bcrypt = Bcrypt(app)

@app.route("/usuarios/registrar", methods=["POST"])
def registrar():
    # Validar datos primero (usando validar_registro)
    if not Usuario.validar_registro(request.form.to_dict()):
        return redirect("/registro")

    # Hashear la contraseña antes de guardar
    pass_hasheado = bcrypt.generate_password_hash(request.form['password'])

    # Crear diccionario con datos (contraseña hasheada)
    datos = {
        "nombre": request.form['nombre'],
        "apellido": request.form['apellido'],
        "email": request.form['email'],
        "password": pass_hasheado, # Guardamos el hash, NO la contraseña
        "edad": request.form.get('edad')
    }

    # Guardar usuario
    nuevo_id = Usuario.save(datos)

    # Guardar ID en sesión
    session['usuario_id'] = nuevo_id

    return redirect("/dashboard")
```

Comparar Contraseña en el Login

```
# Cuando el usuario inicia sesión:
@app.route("/usuarios/login", methods=["POST"])
def login():
    # Buscar usuario por email
    usuario = Usuario.buscar_por_email(request.form['email'])

    if not usuario:
        flash("E-mail no registrado", "login")
        return redirect("/")

    # Comparar contraseña ingresada con hash almacenado
    if not bcrypt.check_password_hash(usuario.password, request.form['password']):
        flash("Contraseña incorrecta", "login")
        return redirect("/")

    # Si todo está bien, guardar ID en sesión
    session['usuario_id'] = usuario.id
    return redirect("/dashboard")
```


¿Qué hace `check_password_hash()` ?

1. Extrae el salt del hash almacenado
2. Aplica el salt a la contraseña ingresada
3. Hashea la contraseña con salt
4. Compara con el hash almacenado

Actualizar el Modelo Usuario

```
# Necesitamos actualizar el modelo para incluir password:

class Usuario:
    def __init__(self, data):
        self.id = data["id"]
        self.nombre = data["nombre"]
        self.apellido = data["apellido"]
        self.email = data["email"]
        self.password = data.get("password") # Campo password
        self.edad = data.get("edad")
        self.created_at = data["created_at"]
        self.updated_at = data["updated_at"]

    @classmethod
    def save(cls, datos):
        # Si password está presente, incluirlo
        if 'password' in datos and datos['password']:
            query = """
                INSERT INTO usuarios
                (nombre, apellido, email, password, edad)
                VALUES (%(nombre)s, %(apellido)s, %(email)s, %(password)s, %(edad)s)
            """
        else:
            query = """
                INSERT INTO usuarios
                (nombre, apellido, email, edad)
                VALUES (%(nombre)s, %(apellido)s, %(email)s, %(edad)s)
            """
        return connectToMySQL('tienda').query_db(query, datos)
```

Actualizar la Base de Datos

Script SQL para agregar campo password:

```
USE tienda;  
  
ALTER TABLE usuarios  
ADD COLUMN password VARCHAR(255) NULL AFTER email;
```

Ejecutar este script antes de usar autenticación.

Nota: Los registros existentes tendrán `password = NULL`. Solo los nuevos usuarios tendrán contraseña.

Validación de Registro con Contraseña

```
# Actualizamos el método de validación:
@staticmethod
def validar_registro(data):
    es_valido = True

    # ... validaciones de nombre, apellido, email ...

    # Validar contraseña
    if len(data.get('password', '').strip()) == 0:
        flash("La contraseña es obligatoria", "password")
        es_valido = False
    elif len(data['password'].strip()) < 8:
        flash("La contraseña debe tener al menos 8 caracteres", "password")
        es_valido = False

    # Validar confirmación de contraseña
    if 'confirm_password' in data:
        if data['password'] != data['confirm_password']:
            flash("Las contraseñas no coinciden", "password")
            es_valido = False

    return es_valido
```

Sistema de Autenticación Completo

Componentes del Sistema

Un sistema de autenticación completo necesita:

1. Registro (Sign Up)

- Formulario con validaciones
- Hash de contraseña
- Guardar en BD

2. Inicio de Sesión (Login)

- Verificar email existe
- Comparar contraseña
- Crear sesión

3. Cierre de Sesión (Logout)

- Eliminar sesión
- Redirigir

4. Protección de Rutas

- Verificar sesión activa
- Redirigir si no hay sesión

Estructura del Proyecto

```
02_modularizacion_crud_usuario/  
├── flask_app/  
│   ├── __init__.py           (con secret_key y bcrypt)  
│   ├── models/  
│   │   └── usuario.py        (con validar_registro, buscar_por_email)  
│   ├── controllers/  
│   │   └── usuarios.py       (registro, login, logout)  
├── templates/  
│   ├── index.html            (formularios registro/login)  
│   ├── dashboard.html        (página protegida)  
│   └── usuarios/  
│       └── nuevo_usuario.html  
└── server.py
```


Configuración Inicial

`flask_app/__init__.py` :

```
from flask import Flask
from flask_bcrypt import Bcrypt

app = Flask(__name__)
app.secret_key = "Mi super clave secreta" # Para sesiones y flash

bcrypt = Bcrypt(app) # Objeto bcrypt global
```

Importante: `secret_key` y `bcrypt` deben estar configurados aquí.

Ruta Raíz: Formularios de Registro y Login

```
<!-- templates/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Registro e Inicio de Sesión</title>
</head>
<body>
  <h1>Registro</h1>
  <form action="/usuarios/registrar" method="POST">
    <input type="text" name="nombre" placeholder="Nombre" required>
    <input type="text" name="apellido" placeholder="Apellido" required>
    <input type="email" name="email" placeholder="Email" required>
    <input type="password" name="password" placeholder="Contraseña" required>
    <input type="password" name="confirm_password" placeholder="Confirmar Contraseña" required>
    <button type="submit">Registrarse</button>
  </form>

  <h1>Inicio de Sesión</h1>
  <form action="/usuarios/login" method="POST">
    <input type="email" name="email" placeholder="Email" required>
    <input type="password" name="password" placeholder="Contraseña" required>
    <button type="submit">Iniciar Sesión</button>
  </form>

  <!-- Mensajes flash -->
  {% with messages = get_flashed_messages() %}
    {% if messages %}
      {% for message in messages %}
        <p>{{ message }}</p>
      {% endfor %}
    {% endif %}
  {% endwith %}
</body>
</html>
```

Controlador: Registro

```
# controllers/usuarios.py
from flask import render_template, redirect, request, flash, session
from flask_app import app, bcrypt
from flask_app.models.usuario import Usuario

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/usuarios/registrar", methods=["POST"])
def registrar():
    # Validar datos
    if not Usuario.validar_registro(request.form.to_dict()):
        return redirect("/")

    # Hashear contraseña
    pass_hasheado = bcrypt.generate_password_hash(request.form['password'])

    # Preparar datos
    datos = {
        "nombre": request.form['nombre'],
        "apellido": request.form['apellido'],
        "email": request.form['email'],
        "password": pass_hasheado,
        "edad": request.form.get('edad')
    }

    # Guardar usuario
    nuevo_id = Usuario.save(datos)

    # Crear sesión
    session['usuario_id'] = nuevo_id
    flash("¡Registro exitoso!", "exito")

    return redirect("/dashboard")
```

Controlador: Login

```
@app.route("/usuarios/login", methods=["POST"])
def login():
    # Buscar usuario por email
    usuario = Usuario.buscar_por_email(request.form['email'])

    if not usuario:
        flash("E-mail no registrado", "login")
        return redirect("/")

    # Verificar contraseña
    if not bcrypt.check_password_hash(usuario.password, request.form['password']):
        flash("Contraseña incorrecta", "login")
        return redirect("/")

    # Crear sesión
    session['usuario_id'] = usuario.id
    flash("¡Bienvenido de nuevo!", "exito")

    return redirect("/dashboard")
```

Flujo:

1. Buscar usuario por email
2. Si no existe → error
3. Comparar contraseña con hash
4. Si no coincide → error
5. Si todo está bien → crear sesión

Controlador: Dashboard (Protegido)

```
@app.route("/dashboard")
def dashboard():
    # Verificar si hay sesión activa
    if 'usuario_id' not in session:
        flash("Debes iniciar sesión para acceder", "error")
        return redirect("/")

    # Obtener usuario actual
    usuario = Usuario.get_one(session['usuario_id'])

    if not usuario:
        session.clear()
        flash("Usuario no encontrado", "error")
        return redirect("/")

    return render_template("dashboard.html", usuario=usuario)
```

Protección: Solo usuarios con sesión activa pueden acceder.

Controlador: Logout

```
@app.route("/usuarios/logout", methods=["POST"])
def logout():
    # Limpiar sesión
    session.clear()
    flash("Sesión cerrada correctamente", "exito")
    return redirect("/")
```

O usando GET:

```
@app.route("/usuarios/logout")
def logout():
    session.clear()
    flash("Sesión cerrada correctamente", "exito")
    return redirect("/")
```

session.clear() : Elimina todos los datos de la sesión.

Plantilla Dashboard

```
<!-- templates/dashboard.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Dashboard</title>
</head>
<body>
  <h1>Bienvenido, {{ usuario.nombre }} {{ usuario.apellido }}!</h1>
  <p>Email: {{ usuario.email }}</p>

  <!-- Mensajes flash -->
  {% with messages = get_flashed_messages() %}
    {% if messages %}
      {% for message in messages %}
        <p>{{ message }}</p>
      {% endfor %}
    {% endif %}
  {% endwith %}

  <!-- Botón de cerrar sesión -->
  <form action="/usuarios/logout" method="POST">
    <button type="submit">Cerrar Sesión</button>
  </form>
</body>
</html>
```


Validaciones Requeridas para Registro

Según el contenido Core, las validaciones son:

1. **Nombre:** Solo letras, al menos 2 caracteres, no vacío
2. **Apellido:** Solo letras, al menos 2 caracteres, no vacío
3. **Email:** Formato correcto, único en BD, no vacío
4. **Contraseña:** Al menos 8 caracteres, no vacío
5. **Confirmación:** Debe coincidir con contraseña

Método `validar_registro()` completo:

```
# models/usuario.py
@staticmethod
def validar_registro(data):
    es_valido = True

    # Nombre: solo letras, al menos 2 caracteres
    if len(data['nombre'].strip()) == 0:
        flash("El nombre es obligatorio", "nombre")
        es_valido = False
    elif len(data['nombre'].strip()) < 2:
        flash("El nombre debe tener al menos 2 caracteres", "nombre")
        es_valido = False
    elif not data['nombre'].strip().replace(' ', '').isalpha():
        flash("El nombre solo puede contener letras", "nombre")
        es_valido = False

    # ... similar para apellido ...

    # Email: formato y unicidad (ya lo tenemos)

    # Contraseña: al menos 8 caracteres
    if len(data.get('password', '').strip()) < 8:
        flash("La contraseña debe tener al menos 8 caracteres", "password")
        es_valido = False

    # Confirmación
    if data['password'] != data['confirm_password']:
        flash("Las contraseñas no coinciden", "password")
        es_valido = False

    return es_valido
```

Validaciones para Login

El login solo necesita verificar:

1. **Email existe** en la base de datos
2. **Contraseña coincide** con el hash almacenado

No necesitamos método de validación separado, solo verificaciones en el controlador:

```
@app.route("/usuarios/login", methods=["POST"])
def login():
    # Verificar email existe
    usuario = Usuario.buscar_por_email(request.form['email'])
    if not usuario:
        flash("E-mail no registrado", "login")
        return redirect("/")

    # Verificar contraseña
    if not bcrypt.check_password_hash(usuario.password, request.form['password']):
        flash("Contraseña incorrecta", "login")
        return redirect("/")

    # Todo correcto
    session['usuario_id'] = usuario.id
    return redirect("/dashboard")
```

Para proteger rutas que requieren autenticación:

```
def login_required(f):
    """Decorador para proteger rutas"""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'usuario_id' not in session:
            flash("Debes iniciar sesión para acceder", "error")
            return redirect("/")
        return f(*args, **kwargs)
    return decorated_function

# Uso:
@app.route("/dashboard")
@login_required
def dashboard():
    usuario = Usuario.get_one(session['usuario_id'])
    return render_template("dashboard.html", usuario=usuario)
```

O verificar manualmente en cada ruta:

```
@app.route("/dashboard")
def dashboard():
    if 'usuario_id' not in session:
```



Resumen de Conceptos Clave

Lo Más Importante

1. **NUNCA** guardar contraseñas en texto plano en la base de datos
2. Hashing es irreversible, perfecto para contraseñas
3. Bcrypt genera salt automáticamente y es intencionalmente lento
4. `generate_password_hash()` crea el hash para guardar
5. `check_password_hash()` compara contraseña con hash
6. Sesiones almacenan el ID del usuario autenticado
7. Proteger rutas verificando sesión activa

Registro:

```
Usuario llena formulario
↓
Validar datos
↓
Hashear contraseña (bcrypt)
↓
Guardar en BD
↓
Crear sesión (usuario_id)
↓
Redirigir a dashboard
```

Login:

```
Usuario ingresa email/password
↓
Buscar usuario por email
↓
Comparar password con hash
↓
Si coincide → crear sesión
```


Conceptos Técnicos

Bcrypt:

- `bcrypt = Bcrypt(app)` : Crear objeto bcrypt
- `generate_password_hash(password)` : Crear hash
- `check_password_hash(hash, password)` : Verificar contraseña

Sesiones:

- `session['usuario_id'] = id` : Guardar en sesión
- `session.clear()` : Limpiar sesión
- `'usuario_id' in session` : Verificar sesión activa

Validaciones:

- `validar_registro()` : Para registro (incluye password)
- `buscar_por_email()` : Para verificar unicidad y login

Buenas Prácticas

- ✓ Siempre hashear contraseñas antes de guardar
- ✓ Usar Bcrypt en lugar de algoritmos antiguos
- ✓ Validar antes de guardar (email único, formato, etc.)
- ✓ Mensajes de error claros pero no revelar información sensible
- ✓ Proteger rutas sensibles verificando sesión
- ✓ Limpiar sesión al cerrar sesión
- ✓ Usar HTTPS en producción (no guardar contraseñas sin cifrado)

Errores Comunes a Evitar

- ✗ Guardar contraseñas en texto plano → Riesgo de seguridad crítico
- ✗ No verificar sesión en rutas protegidas → Acceso no autorizado
- ✗ Mensajes de error muy específicos → Ayudan a hackers
- ✗ Olvidar limpiar sesión → Usuario sigue "logueado"
- ✗ No validar confirmación de contraseña → Errores de tipeo
- ✗ Usar algoritmos antiguos (MD5, SHA1) → Vulnerables

Recursos para Seguir Aprendiendo

Documentación Oficial

-  [Flask-Bcrypt Documentation](#)
-  [Flask Sessions](#)
-  [Bcrypt Algorithm](#)

Próximos Pasos

En la siguiente clase aprenderemos:

-  Relaciones uno a muchos con autenticación
-  Foro de publicaciones con usuarios autenticados
-  Asociar datos con usuarios (publicaciones, comentarios)
-  Mejorar la experiencia del usuario autenticado

Práctica para Casa

Proyecto sugerido:

Crea una aplicación de **gestión de tareas** con autenticación:

1. Sistema de registro e inicio de sesión completo
2. Cada usuario ve solo sus propias tareas
3. Proteger rutas para que solo usuarios autenticados puedan crear/editar tareas
4. Implementar logout
5. **BONUS:** Agregar validación de contraseña fuerte (mayúscula, número, carácter especial)

 ¡Felicidades!

Ya sabes proteger contraseñas y crear autenticación

Has aprendido seguridad esencial para aplicaciones web 

? Preguntas

¿Alguna duda sobre autenticación y seguridad?

 ¡Excelente Trabajo!